# Arquitectura basada en representación intermedia para la generación confiable de consultas SQL/PGQ desde lenguaje natural

## Plan de tesis – Avance parcial

Guido Luis Tapia Oré
Asesora: Dra. Regina Paola Ticona Herrera

Universidad Católica San Pablo
Escuela Profesional de Ciencia de la Computación

2026

# Contenido

Motivación y problema

Objetivos

Estado del arte

Propuesta

Representación intermedia: IR-SQL/PGQ

Plan experimental y avance

Conclusiones

# Motivación: interfaces en lenguaje natural sobre datos

**Tendencia actual.** Las organizaciones *data-driven* migran de interfaces gráficas a interfaces conversacionales sobre sus bases de datos (Adedeji & Mabere, 2023; Quamar et al., 2022).

**Tres pilares para producción:**

- **Integridad técnica:** ejecutabilidad y fidelidad semántica al esquema (Lei et al., 2025).
- **Seguridad y gobernanza:** control de acceso y auditoría.
- **Robustez al esquema:** estabilidad ante

## Brecha observada

El alto desempeño en *benchmarks* clásicos decae drásticamente sobre bases corporativas reales (Lei et al., 2025; J. Li et al., 2023).

**Adicionalmente:** la estandarización ISO/IEC 9075-16:2023 (ISO/IEC JTC 1/SC 32, 2023) integra *property graphs* dentro de SQL (**SQL/PGQ**), expandiendo el espacio del problema.

# El problema: alucinaciones en *text-to-query* sobre SQL/PGQ

**Causa estructural.** Los LLM predicen tokens sin mecanismos intrínsecos que vinculen la salida con el esquema ni con la semántica del lenguaje objetivo. Esta desalineación produce **alucinaciones estructurales**:

- Tablas o columnas inexistentes; rutas de *join* inventadas.
- Condiciones lógicas inconsistentes; agregaciones incongruentes.
- Patrones de grafo mal formados; etiquetas de vértice o arista inventadas.

**Agravante en SQL/PGQ.** La consulta opera sobre *dos estructuras superpuestas*:

1. esquema relacional (tablas, columnas, claves, tipos);
2. grafo de propiedad (vértices, aristas, etiquetas, propiedades).

La incoherencia cruzada relacional–grafo introduce **nuevas categorías de fallo** no presentes en *text-to-SQL* clásico.

## Pregunta de investigación

¿Cómo evitar de forma sistemática las alucinaciones estructurales semánticas en la

# Objetivos

**Objetivo general.** Proponer una arquitectura de razonamiento estructural basada en representación intermedia (RI) para la generación *text-to-query* orientada al estándar SQL/PGQ, que mitigue las alucinaciones de esquema y los errores semánticos en escenarios de consulta empresarial.

**Objetivos específicos.**

1. **Caracterizar** los tipos de alucinación en *text-to-query* mediante una taxonomía aplicable a consultas relacionales e híbridas.
2. **Diseñar** una RI unificada (IR-SQL/PGQ) con reglas de tipado y compilación determinista.
3. **Implementar** un *pipeline* con anclaje al esquema, generación restringida y bucle de verificación estática y dinámica.
4. **Evaluar** la propuesta frente al estado del arte mediante métricas de corrección, alucinación, robustez y eficiencia sobre *benchmarks* establecidos.

# Estado del arte: panorama y brecha

| **Familia** | **Aporte principal** | **Limitación frente a SQL/PGQ híbrido** |
|---|---|---|
| Decodificación restringida (Scholak et al., 2021) | Validez sintáctica por *token* | Opera a nivel léxico; no captura coherencia cruzada relacional–grafo. |
| Descomposición y agentes (Pourreza & Rafiei, 2023; Wang et al., 2024) | Divide la generación en sub-tareas | Restringido al régimen relacional puro. |
| Alineación pre-generación (Qu et al., 2024) | Reduce alucinaciones por anclaje al esquema | No contempla esquemas duales. |
| *Schema linking* y M-Schema (Cao et al., 2024; Gao et al. | Representación rica y enlazado bidireccional | Solo describe el bloque relacional. |

# Bases técnicas seleccionadas

**Bases técnicas seleccionadas y cobertura taxonómica**

**XiYan-SQL**
representación M-Schema
Representación

*referencia de diseño*

**CHESS**
arquitectura modular
Arquitectura
Enriquecimiento
Razonamiento

*descripción de mecanismo*

**RSL-SQL**
enlazado al esquema
Enriquecimiento

*descripción de mecanismo*

**DuckDB + DuckPGQ**
motor SQL/PGQ
Evaluación

**NL2SQL360**
marco de evaluación
Evaluación

**sqlglot**
análisis sintáctico SQL
Razonamiento

# Requisitos y principios de diseño

## Requisitos (R1–R6)

- **R1** Anclaje al esquema dual.
- **R2** Representación intermedia tipada.
- **R3** Separación generación / compilación.
- **R4** Verificación en dos regímenes.
- **R5** Modularidad y agnosticismo del LLM.
- **R6** Trazabilidad de decisiones.

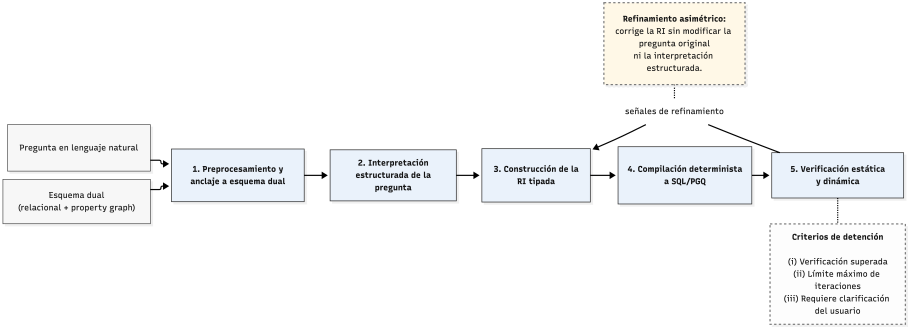
## Principios (P1–P5)

- **P1** *Grounding* antes que generación.
- **P2** Mediación obligatoria por RI.
- **P3** Verificación en cascada (estática → dinámica).
- **P4** Transparencia estructural y auditabilidad.
- **P5** Desacoplamiento del modelo generador.

### Idea central

Las alucinaciones se confinan al espacio de la *RI tipada*, donde son **detectables y rechazables** antes de alcanzar el motor de ejecución.

# Pipeline propuesto




1. **Pre-procesamiento y anclaje al esquema dual.** Construye $\Sigma_{\text{proj}} = \langle T_{\text{rel}}, V_{\text{grafo}}, E_{\text{grafo}}, C_{\text{val}} \rangle$.
2. **Interpretación estructurada** de la pregunta: entidades, restricciones, tipo de razonamiento.
3. **Construcción de la RI tipada** (IR-SQL/PGQ).
4. **Compilación determinista** a SQL/PGQ.
5. **Verificación** estática y dinámica, con bucle de refinamiento sobre la RI.

# IR-SQL/PGQ – Estructura general

**Una RI en tres bloques cerrados.**

## Bloque relacional

- `TableRef`, `ColumnRef`.
- Expresiones recursivas, agregaciones, sub-consultas.
- `RelationalQuery`, `SetOperation`.

*Cobertura empírica: 99 % round-trip estructural sobre Spider dev.*

## Bloque de grafo

- `VertexPattern`: variable + etiqueta.
- `EdgePattern`: variable + etiqueta + dirección.
- `PathPattern`: caminos *multihop*.
- `MatchPattern`: raíz, expone columnas.

*Materializa la cláusula* `GRAPH_TABLE(MATCH ...COLUMNS)` *del estándar* (ISO/IEC JTC 1/SC 32, 2023; Wolde et al., 2023).

## Composición híbrida

- `FromGraphMatch` ocupa cualquier posición de `FromExpr`.
- Composicionalidad estructural: sin nodos especiales para el caso mixto.
- Tres ámbitos separados: aliases relacionales, aliases de grafo, variables del MATCH.

## Linaje del diseño

**SonSQL** (Cuenca et al., 2010), [illegible] **NetSQL**

# IR-SQL/PGQ – Contrato JSON e invariantes operativos

**Contrato bidireccional con el LLM.**

- Cada nodo es una *dataclass* `frozen` → objeto JSON con campo `type`.
- JSON Schema *draft 2020-12* se inyecta como `input_schema` de una *tool* en la API del modelo.
- Cuatro **guardas en serie**: *schema* → *parser* → verificador estático → verificación dinámica.
- El modelo **no puede emitir SQL libre**: la única salida válida es una instancia de la RI.

**Cuatro invariantes validados empíricamente.**

1. *Inmutabilidad e igualdad estructural* (dataclasses `frozen`).
2. *Round-trip lift–compilación*: el compilador es inversa derecha del lifter.
3. *Round-trip parse–serialización*: `parse_ir(to_dict(q)) == q`.
4. *Soundness referencial*: toda referencia válida en la RI corresponde a $\Sigma_{\mathrm{proj}}$.

**Diferencia frente a auto-corrección estándar**

La corrección se aplica *ex ante* sobre la RI, antes de que cualquier SQL textual exista, no *ex post* sobre la salida del motor.

# Verificación: estática y dinámica

**Verificación estática** sobre IR-SQL/PGQ (cuatro familias):

- **Referencial**: `unknown_table`, `unknown_column`, `unknown_vertex_label`, `unknown_edge_label`.
- **Consistencia de tipos**: `type_mismatch_aggregate`, `type_mismatch_arithmetic`.
- **Coherencia cruzada relacional–grafo**: `vertex_label_without_table`, `path_step_incoherent`.
- **Restricciones operativas del motor** (DuckPGQ): `missing_vertex_label`.

**Verificación dinámica.** Si la consulta supera el chequeo estático, se ejecuta sobre DuckPGQ (Wolde et al., 2023) como *oráculo parcial*: válida, inválida por error, o ejecutable con salida atípica.

## Política de retroalimentación estructurada

El bucle no reenvía al LLM el mensaje crudo del motor, sino un **descriptor categórico** que se inyecta en la etapa de construcción de la RI. Refuerza P4 (auditabilidad) y P2 (mediación tipada).

# Plan experimental

## Stack técnico fijado.

- **Motor:** DuckDB 1.4.4 + extensión `duckpgq`.
- **LLM inicial:** Claude Haiku 4.5 (sustituible por P5).
- **Parser SQL:** sqlglot, dialecto `duckdb`.
- **Entorno:** `uv` 0.11+, Python 3.11.

## Datasets y benchmarks.

- **Spider** (Yu et al., 2018), **BIRD** (J. Li et al., 2023): régimen relacional.
- **Spider 2.0** (Lei et al., 2025): régimen empresarial.
- **Corpus PGQ propio**: régimen híbrido SQL/PGQ.

## Métricas.

- **Canónicas** (B. Li et al., 2024): *Execution Accuracy* (EX), *Exact Match* (EM), *Valid Efficiency Score* (VES).
- **Propias para SQL/PGQ**: tasa de alucinación referencial, coherencia cruzada, descriptores de error categóricos.

## Comparación.

- *Baselines* arquitectónicos: CHESS (Talaei et al., 2024), XiYan-SQL (Gao et al., 2024).
- *Ablations*: con/sin RI, con/sin verificación dinámica, con/sin coherencia cruzada.

# Avance actual

**Componentes implementados.**

- Definición de las *dataclasses* de IR-SQL/PGQ y JSON Schema asociado.
- Compilador determinista IR $\rightarrow$ SQL/PGQ con generación sistemática de variables de arista (restricción operativa de DuckPGQ).
- Verificador estático con las cuatro familias de chequeo y descriptores canónicos.
- *Lifter* de SQL a RI: 99 % *round-trip* estructural sobre 100 consultas predichas en Spider *dev*.

**Experimentos preliminares.**

- **Exp. 07**: comparativa Haiku 4.5 vs. Sonnet 4.6 sobre el *baseline*.
- **Exp. 08–09**: bucle de retroalimentación estructurada con clasificador de errores; corpus PGQ y Spider.

## Hallazgo cualitativo temprano

La mediación por RI tipada produce errores *categorizables y reproducibles*, en contraste con la opacidad de los esquemas de auto-corrección sobre SQL textual.

# Cronograma – hitos principales

| Fase | Hito | Periodo | Avance |
|---|---|---|---|
| Parcial 26-1 | Plan de tesis: capítulos 1–4 + cronograma | Mar–May 2026 | 55 % |
| | Implementación arquitectura v1 (NL → RI → consulta) | May–Jun 2026 | 68 % |
| | Experimentación preliminar y exposición de avances | Jun 2026 | 80 % |
| Inter. | Refinamiento arquitectónico y revisión bibliográfica | Jul–Ago 2026 | 82 % |
| 26-2 | Ampliación experimental, ablations, análisis de errores | Ago–Oct 2026 | 91 % |
| | Redacción final, sustentación | Nov–Dic 2026 | 100 % |

**Estado actual**: cierre de Plan de Tesis y diapositivas de exposición parcial.

# Conclusiones provisionales y trabajo futuro

**Conclusiones provisionales.**

- Existe una brecha clara: las técnicas de mitigación de alucinaciones están *fragmentadas* y no cubren el régimen híbrido SQL/PGQ.
- La **mediación por RI tipada** traslada al espacio de la representación los errores que actualmente se detectan tarde (o no se detectan) sobre SQL textual.
- El prototipo demuestra factibilidad técnica: lifter, compilador determinista y verificador estático integrados sobre DuckPGQ.

**Trabajo futuro.**

- Cierre del corpus PGQ propio para evaluación canónica del régimen híbrido.
- Ampliación de la cobertura de la RI (*window functions*, mecanismos de ambigüedad inspirados en Ding et al., 2025).
- *Ablations* sistemáticos sobre el bucle de verificación.
- Evaluación con LLMs adicionales (Sonnet 4.6, GPT, Gemini) bajo P5.

# Gracias por su atención.

Quedo a su disposición para preguntas y observaciones.

*Guido Luis Tapia Oré – UCSP – 2026*

# Referencias I


Adedeji & Mabere, M. (2023). From Data to Decisions: LLMs Redefining Business Intelligence. *Journal of Artificial Intelligence, 6*.

Cao, Z., et al. (2024). RSL-SQL: Robust Schema Linking in Text-to-SQL Generation. https://arxiv.org/abs/2411.00073

Ding, Z., Lin, Y., Zeng, T., et al. (2025). AmbiSQL: Interactive Ambiguity Detection and Resolution for Text-to-SQL. https://arxiv.org/abs/2508.15276

Gao, Y., Liu, Y., Li, X., et al. (2024). XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL. https://arxiv.org/abs/2411.08599

Guo, J., Zhan, Z., Gao, Y., Xiao, Y., Lou, J.-G., Liu, T., & Zhang, D. (2019). Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. https://arxiv.org/abs/1905.08205

# Referencias II


ISO/IEC JTC 1/SC 32. (2023, junio). *Information technology – Database languages – SQL – Part 16: Property Graph Queries (SQL/PGQ)* (Standard). International Organization for Standardization. Geneva, CH. https://www.iso.org/standard/79473.html

Lei, F., Su, J., Chen, Y., et al. (2025). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *ICLR*. https://arxiv.org/abs/2411.07763

Li, B., Luo, Y., Chai, C., et al. (2024). The Dawn of Natural Language to SQL: Are We Fully Ready? https://arxiv.org/abs/2406.01265

Li, H., Zhang, J., Li, C., & Chen, H. (2023). RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. *AAAI*. https://arxiv.org/abs/2302.05965

# Referencias III


Li, J., Hui, B., Qu, G., et al. (2023). Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *NeurIPS*. https://arxiv.org/abs/2305.03111

Pourreza, M., & Rafiei, D. (2023). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *NeurIPS*. https://arxiv.org/abs/2304.11015

Qu, G., Li, J., Li, B., et al. (2024). Before Generation, Align it! A Novel and Effective Strategy for Mitigating Hallucinations in Text-to-SQL Generation. *ACL Findings*. https://arxiv.org/abs/2405.15307

Quamar, A., Efthymiou, V., & Lei, C. (2022). Natural Language Interfaces to Data. *Foundations and Trends in Databases*, *11*, 319-414. http://dx.doi.org/10.1561/1900000078

Scholak, T., et al. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *EMNLP*. https://arxiv.org/abs/2109.05093

# Referencias IV


Stoisser, J. L., Martell, M. B., et al. (2025). STRuCT-LLM: Unifying Tabular and Graph Reasoning with Reinforcement Learning for Semantic Parsing. *ICLR*. https://openreview.net/forum?id=xZDoGrMTGI

Talaei, S., Pourreza, M., Chang, Y.-C., et al. (2024). CHESS: Contextual Harnessing for Efficient SQL Synthesis. *ICML*. https://arxiv.org/abs/2405.16755

Wang, B., Ren, C., Yang, J., et al. (2024). MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. *COLING*. https://arxiv.org/abs/2312.11242

Wolde, D. t., Szárnyas, G., & Boncz, P. (2023). DuckPGQ: Bringing SQL/PGQ to DuckDB. *Proc. VLDB Endow.*, *16*(12), 4034-4037.

Yu, T., Zhang, R., Yang, K., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP*. https://arxiv.org/abs/1809.08887